实验二：语法分析实验

一、实验目的

编制SLR（1）分析程序，或LL(1)分析程序，或递归下降分析程序，分析输入程序，加深对语法分析的理解。

二、实验内容

（1）给出上下文无关文法；

（2）对所描述的文法，根据需要，可能构造SLR(1)分析表或LL(1)分析表；

（3）编制语法分析程序；如果输入的源代码没有语法错误，输出Pass，否则输出Error。

文法（1）

```
Program         ::= { FunctionDef | Statement }

FunctionDef     ::= "func" [ "[]" ] Identifier "(" [ ParameterList ] ")" Block

ParameterList   ::= Identifier { "," Identifier }
```

文法（2）

```
Statement        ::= AssignmentStmt
                   | IfStmt
                   | WhileStmt
                   | JumpStmt
                   | DeclarationStmt
                   | Block
                   | ";"

Block            ::= "{" { Statement } "}"

IfStmt           ::= "if" "(" Expression ")" Block "else" Block

WhileStmt        ::= "while" "(" Expression ")" Block

AssignmentStmt   ::= LValue "=" Expression ";"
                   | Identifier "[]" "=" Expression ";"  /* 数组声明或初始化 */
JumpStmt          ::= "break" ";"
                    | "continue" ";"
                    | "return" Expression ";"

DeclarationStmt ::= ( "local" | "global" ) Identifier [ "[]" ] ";"
```

文法（3）

```
LValue            ::= Identifier [ "[" Expression "]" ]
                    | "write"
                    | "put"

Expression        ::= LogicalExp

LogicalExp        ::= RelationalExp { ("==" | "!=") RelationalExp }

RelationalExp     ::= AdditiveExp { ("<" | "<=" | ">" | ">=") AdditiveExp }

AdditiveExp       ::= MultiplicativeExp { ("+" | "-") MultiplicativeExp }

MultiplicativeExp ::= Primary { ("*" | "/" | "%") Primary }

Primary           ::= Integer
                    | CharConstant
                    | StringConstant
                    | Identifier "(" [ ArgumentList ] ")"  /* 函数调用 */
                    | LValue
                    | "read"
                    | "get"
                    | "(" Expression ")"

ArgumentList      ::= Expression { "," Expression }
```

文法处理

$$expr \rightarrow expr + term \mid expr - term \mid term$$

P612 A.1

expr → term { (“+” | “-”) term }

```
Expr expr() throws IOException {
   Expr x = term();
   while( look.tag == '+' || look.tag == '-' ) {
      Token tok = look;  move();  x = new Arith(tok, x, term());
   }
   return x;
}
```

P627 A.8